

БУТКЕМП М.ВИДЕО · NLP / SEARCH · ИТОГИ

Умный поиск: от данных до каскада

Как мы научились понимать поисковый запрос: разобрали данные, собрали MVP, починили опечатки, проверили модели и выложили сервис с честными метриками.

Итоговая презентация · Дни 02–05

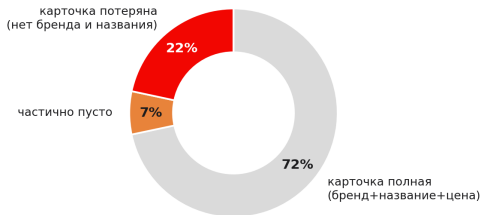
ДЕНЬ 02

Качество данных и постановка

Дыры в датасете, транслит, раскладка, дубли брендов — и почему задача не «обычный NER».

Что нашли, когда прочитали данные руками

Полнота карточки в кликах



ВЫВОД Датасет собирал человек — в нём есть системные дыры и ошибки. Их надо чистить до обучения, иначе модель учится на мусоре.

- 1** Только **72%** кликов имеют полную карточку (бренд + название + цена).
- 2** У **22%** карточка вообще не подтянулась — пусты и бренд, и название (join не срослось при сборке).
- 3** Формальных NaN нет — дыры спрятаны в пустых строках "", их легко не заметить.

КАЧЕСТВО ДАННЫХ · ПЕРЕПУТАННЫЕ КОЛОНКИ

Артикул уехал в поле бренда

ВЫВОД В 66 строках **бренд** — это просто число. В 98,5% случаев это артикул из середины названия, попавший не в ту колонку.

Название товара (sku_name)	Поле «бренд»	Реальность
Вентилятор напольный 27942 GL 8112	27942	артикул
Камера видеонаблюдения 360 V380 Pro APP	360	артикул
Туфли женские 0 Туфли	0	заглушка
Бритва 4088 Extra Smooth Embrace	4088	артикул

Правило для нас: числовой sku_brand_name — не бренд, а шум. Реальный бренд ищем в тексте названия либо считаем, что его нет.

Транслит, смесь языков и омоглифы

Масштаб проблем данных (% от семпла)



ВЫВОД 35% уникальных запросов — смесь кириллицы и латиницы. Модель обязана быть устойчива к раскладке.

Тип	Пример из данных
Смесь	«iPhone 17 про Макс», «bosh лодильник»
Транслит	«noutbuk», «televizor», «Stiralnay»
Омоглиф	«Ресанта» (а — кир.), «Фaza»
Раскладка	«dfhjxyfz» → «варочная»

КАЧЕСТВО ДАННЫХ · ЗАБЫТАЯ РАСКЛАДКА

Печатали по-русски в EN-раскладке

Вывод Нашли **80+** запросов вида «dfhjxyfz gfytkm» — это «варочная панель», набранное в латинской раскладке. Поиск обязан их авто-исправлять.

Ввели (EN)

lkz djkc aty gkjqr f gkjqr
 pfhlyst ecnhjqcndf lkz
 frrevkjnjhd fdnj
 dfhjxyfz gfytkm ,tkjt cntrkh
 dbltjrfvthf lkz gjvtotybz
 dcnhfbdfvzf cnbhfkmyfz
 vfiby f
 le[jdq irfa 'ktrnhbxtcrbq
 pfhzyjt ecnhjqcndj czjvb
 rf,tkm lkz rkdbfnehs
 gsktcjc dthnbrfkmysq
 ghjdilyjq
 nhbvvth lkz yjcf
 'ktrnhbxtcrfz gkbnf akfvf
 ,tjdfz ljhj;rf

Хотели найти

для волос фен плойка плойка
 зарядные устройства для
 аккумуляторов авто
 варочная панель белое стекл
 видеокамера для помещения
 встраиваемая стиральная
 машина
 духовой шкаф электрический
 зарядное устройство сяоми
 кабель для клавиатуры
 пылесос вертикальный
 проводной
 триммер для носа
 электрическая плита флама
 беговая дорожка

Ввели (EN)

dtynbkznjh yfcnjkmysq
 dthnbrfkmysq gsktcjc
 dcnhfbdbdfvzf [kjlbkmybr
 dcnhjtyyfz gjceljvjqr
 dcnhjtyysq [kjlbkmybr
 ufjpdfz gkbnf
 ufqrjdthn htldthu
 ukflbkmyfz ljcrf
 lhtkm ,eh
 pfdfhjxysq xfyqybr
 pfhzyjt ecnhjqcndj
 buhjdjt rhtckj
 buhjdj hek
 rkdbfnehf ,tkfz

Хотели найти

вентилятор настольный
 вертикальный пылесос
 встраиваемый холодильник
 встроенная посудомойка
 встроенный холодильник
 газовая плита
 гайковерт редверг
 гладильная доска
 дрель бур
 заварочный чайник
 зарядное устройство
 игровое кресло
 игровой руль
 клавиатура белая

КАЧЕСТВО ДАННЫХ · ДУБЛИ И МЕРЫ

Дубли брендов и что мы с этим делаем

Один бренд — много написаний

Проблема	Пример
Регистр	APPLE / Apple, honor / HONOR
Перевод	Stinol / Стинол, KION / КИОН
Дубли строк	один клик записан по 2–3 раза

Меры (слой нормализации)

- 1 Приводим бренды к канону: регистр, транслит, схлопывание дублей (**Стинол**=**Stinol**).
- 2 Фильтруем мусор до обучения: пустые карточки и цена ≤ 0 — вон или в отдельную метку.
- 3 Позиция = 0 — это «нет данных», не топ выдачи; не используем как сигнал.
- 4 Это прямой аргумент за **H5**: одним regex единообразие не навести.

205 групп брендов различаются лишь регистром/пробелами, ~209 групп — кандидаты на омоглифы.

ПОСТАНОВКА · ПЕРЕОСМЫСЛЕНИЕ

Задача — не «обычный NER»

ВЫВОД Главная цель — максимизировать покрытие edge-cases, а не выбить красивый F1 на простых запросах.

Почему так

- 1 Голова запросов решается словарём за день — это не подвиг.
- 2 Конверсию теряют хвосты: опечатки, редкие бренды, склейки, смесь языков.
- 3 Ценность решения = сколько хвоста мы покрываем при $SLA < 100$ мс.

Работаем в вероятностном пространстве

Каждый слой отдаёт не только ответ, но и уверенность. Если уверенность низкая — запрос уходит на слой тяжелее. Так простое остаётся быстрым, а сложное — решаемым.

АРХИТЕКТУРА · СЛОИ

Три слоя: голова, середина, хвост

ГОЛОВА

regex + правила +
словари

частые и простые
запросы; мгновенно,
прозрачно, дёшево

низкая увер.

СЕРЕДИНА

CRF и аналоги

token-level: BIO и
другие схемы, либо
переход к span-level
разметке

низкая увер.

ХВОСТ

трансформеры +
LLM **ИЛИ** RNN

edge-cases, которые
не решаются
правилами и обычной
NER-моделью

ВЫВОД Каскад по уверенности: 90% трафика решают дешёвые слои, тяжёлые модели работают только там, где реально нужны.

ЧЕСТНЫЕ МЕТРИКИ · ДЫМОВОЙ ТЕСТ

CRF на слабой BIO-разметке: ранний smoke-тест

ВЫВОД Метрики на серебряных метках оптимистичны — это ориентир, а не финальная оценка. Сейчас есть gold-holdout и журнал метрик; полный gold доразмечаем приложением.

Быстрый тест на выборке

Token accuracy	0.89
Entity micro-F1	0.85
F1 BRAND	0.95
F1 CATEGORY	0.78
F1 ATTR	0.85

Что это значит

- 1** BRAND учится легко (0.95) — словарь почти решает задачу в голове каскада.
- 2** CATEGORY слабее (0.78) — формулировки разнообразнее, тут нужен либо трансформер, либо больше слабо-размеченных данных.
- 3** Итог: CRF — рабочая середина каскада, но категорию и хвост дотягиваем следующим слоем.

EDA · ПРОВЕРКА ГИПОТЕЗ

Гипотезы: что подтвердилось

Гипотеза	Статус	Комментарий
Н1. Запросы — короткие категорийные интенды	Да	топы — чистые категории
Н2. Клики концентрируются в топе выдачи	Да	медиана позиции = 4
Н3. Бренд в запросе ⇒ клик того же бренда	В основном	heatmap диагональна
Н4. Цена сильно влияет на позицию клика	Нет	hexbin: связь слабая
Н5. Атрибуты пишут единообразно	Нет	«16гб», «16 gb», «16 ГБ»
Н6. Бренд обычно написан в запросе	Нет	только 27% — нужен классификатор

Красное = подтверждено данными. Н5 — атрибуты пишут как попало: одним regex не закрыть, нужен отдельный классификатор типов.

ОБНОВЛЕНИЕ · РАЗМЕТКА

Что тогда уже работало: лемматизация

ВЫВОД К тому моменту подключили лемматизацию (Natasha) — словарь начал ловить любые словоформы: «холодильникАМИ», «краснУЮ», «стиральнУЮ машинУ».

Что уже умели

- 1 Лемматизация: «холодильникАМИ» становится «холодильник» — категория находится, даже если слово склонено. Это прямо чинило слабый CATEGORY-recall.
- 2 Атрибутов редех стало **18** вместо 7: память, размер, мощность, ток, вес, объём, частота, гарантия и др.
- 3 Расклейка склеек: «16гб» превращается в «16 гб», спаны атрибутов стали точными.

Пример разбора

запрос: «стиральную машину haier белую»

бренд: **Haier**

категория: **стиральная машина**

атрибуты: цвет = **белую**

Как держим SLA < 100 мс с лемматизацией

ВЫВОД Нейромодель Natasha грузится 1 раз при старте (~1 с), а сам разбор запроса — <1 мс. Онлайн-лемматизация в SLA укладывается, но голову трафика кэшируем.

Замеры (наш семпл)

Загрузка модели (старт)	~965 мс
Лемматизация 1 запроса	~0.6 мс
Токенизация	0.007 мс
Бюджет SLA	100 мс

Модель загружается один раз при инициализации сервиса, а не на каждый запрос — поэтому старт не влияет на латентность ответов.

Что кэшируем / планируем

- 1 Леммы словарей** — считаем один раз офлайн при сборке лейблера, а не на каждый запрос.
- 2 Результат по запросу** (LRU-кэш): голова трафика — это одни и те же запросы; 90% кликов дают лишь 60% уникальных строк.
- 3 Леммы токенов** — кэш «словоформа → лемма»: слова повторяются, второй раз не считаем.
- 4 Фолбэк:** без Natasha «голова» каскада (regex + словарь) работает — деградируем мягко.

ДЕНЬ 03

MVP: предобработка, MODEL, каскад

Единый слой очистки, тег MODEL, brand_clf, приложения поиска и разметки.

Единый слой очистки запроса

Шесть шагов очистки

Шаг	Что делает
Базовая чистка	убирает мусорные пробелы, ё → е
Расклейка	«128гб» → «128 гб»
Сепараторы	«g-pro» → «g pro»
Нормализация	единый ключ для словарей
Защита брендов	«Красный Октябрь» — не цвет
Подсказки	спаны моделей поверх разметки

Зачем один общий слой

- 1 Раньше каждый ноутбук чистил запрос по-своему — результаты не сходились. Теперь очистка одна на всех.
- 2 Регистр сохраняем там, где он несёт смысл: «красный» — это цвет, а «Красный Октябрь» — бренд конфет.
- 3 Модуль отдаёт и чистый текст, и подсказки: где в запросе модель, а где защищённый бренд.

ЭФФЕКТ Спан модели появляется у 18% запросов, склейки чинятся у 11%, а каждый четвёртый «пустой» хвост после бренда оживает.

ПРЕДОБРАБОТКА · ПРИМЕР

Живой пример: запрос до и после

ИСХОДНЫЙ ЗАПРОС «Наушники Logitech G-Pro X SE 128гб» — дефисы, клейки, смесь языков.

Шаг 1 · расклейка и сепараторы

наушники logitech g pro x se 128 гб

«G-Pro» распался в «g pro», «128гб» — в «128 гб». Теперь словари найдут каждую часть.

Шаг 2 · разметка со словарями и подсказками

наушники · КАТЕГОРИЯ

logitech · БРЕНД

g pro x se · МОДЕЛЬ

128 гб · АТРИБУТ

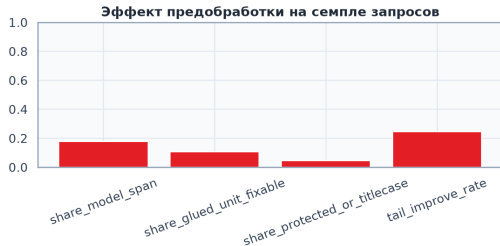
«g pro x se» поймал словарь моделей из майнинга, «128 гб» — регулярка памяти.

Шаг 3 · готовые факты

бренд = Logitech · категория = наушники · модель = g pro x se · память = 128 ГБ.

ПРЕДОБРАБОТКА · ИЗМЕРЕНИЕ

Что даёт предобработка на семпле



КАК МЕРИЛИ Взяли 12 000 уникальных запросов и прогнали через пайплайн дважды: со словарём моделей и без него.

- 1** 18% запросов получают спан модели — примерно каждый пятый.
- 2** 11% содержат склейку вида «128гб», которую чинит расклейка.
- 3** 24% пустых хвостов после бренда получают теги модели после подсказок.
- 4** Защита брендов срабатывает у 4,5% — редко, но спасает «Красный Октябрь» от превращения в цвет.

ТЕГ MODEL · ПРОБЛЕМА

Хвост после бренда теряется



ПРОБЛЕМА После бренда часто идёт название линейки — «dyson v15», «galaxy s24». При слабой разметке эти слова остаются без тега, и CRF не может их выучить.

- 1 Из 15 тысяч запросов бренд есть у 7046. Почти у половины (46%) после бренда остался неразмеченный латинский хвост.
- 2 Типичная длина хвоста — два токена: «zenbook a16», «ipad air».
- 3 Это не атрибут — нет числа с единицей. Нужен отдельный тег MODEL и словарь линеек.

ТЕГ MODEL · ПРИМЕРЫ

Как выглядят потерянные хвосты

ДО СЛОВАРЯ Серые слова — то, что разметка не смогла объяснить. Именно они и есть модели.

До: хвост без тега

пылесос dyson v15
телефон samsung galaxy s24
asus zenbook a16
apple ipad air

После: словарь линеек подключён

пылесос dyson v15 · МОДЕЛЬ
телефон samsung galaxy s24 · МОДЕЛЬ
asus zenbook a16 · МОДЕЛЬ
apple ipad air · МОДЕЛЬ

ЧТО ИЗМЕНИЛОСЬ Хвост после бренда перестал быть «мусором» — теперь это полноценный факт «модель», по которому можно искать и ранжировать.

Майнинг словаря: пример «g pro»

ИДЕЯ Названия кликнуток уже содержат модели. Находим в названии бренд — и всё, что идёт после него, считаем кандидатом в модели.

Название клинута → кандидаты

Наушники

Logitech

G

PRO

X

SE

Black

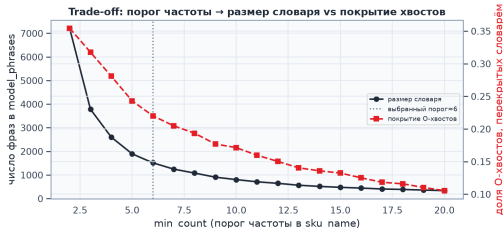
После бренда снимаем 1–4 коротких токена:

Кандидат	Раз
g	213
g pro	87
g pro x	31
g pro x se	9
g pro x se black	2 — отсеки

Почему порог — 6 раз

- 1 Фраза попадает в словарь, если встретилась **минимум 6 раз**. «g pro» видели 87 раз — берём. «g pro x se black» — два раза, случайный хвост, отсекаем.
- 2 Порог 2 — словарь раздувается до 7215 фраз и тащит мусор. Порог 20 — остаётся 335 фраз, теряются редкие линейки.
- 3 Шесть — золотая середина: 1512 фраз, 22% хвостов покрыто, словарь чист на 96,6%.

Выбор порога: размер словаря против покрытия

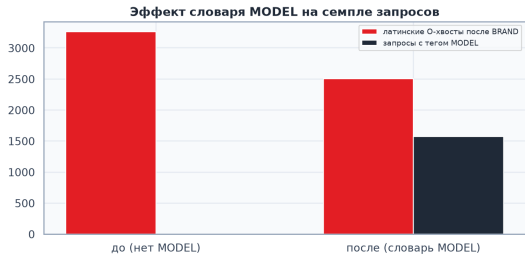


КАК ЧИТАТЬ ГРАФИК Чёрная кривая — размер словаря, красная — доля хвостов, которые он покрывает. Пунктир — наш выбор.

- 1** Из 47 тысяч названий карточек намайнили 8540 сырых кандидатов: galaxy, iphone, watch, redmi и другие.
- 2** Кривые расходятся плавно, «локтя» нет — выбрали порог 6 как компромисс между размером и покрытием.
- 3** Дополнительные фильтры: голые числа не берём; фразы из одного токена — только коды вида «v15», «a305», «ps5».

ТЕГ MODEL · РЕЗУЛЬТАТ

До и после подключения словаря



ВЫВОД Словарь линеек включает класс MODEL: было 0 запросов с тегом — стало 1573. Потерянных хвостов стало меньше на четверть.

- 1 Латинские хвосты без тега: 3262 → 2502, минус 23%.
- 2 Проверили качество словаря вручную: 96,6% фраз — настоящие линейки, остальное — длинные артикулы и аксессуары, их вычистим.
- 3 Оставшиеся хвосты словарём не закрыть — это кандидаты на ручную разметку, для неё мы сделали приложение (покажем дальше).

РАЗМЕТКА · СХЕМЫ

Какие бывают схемы токен-разметки

КОНТЕКСТ Задача NER — найти спаны сущностей. Но модели классифицируют токены, поэтому спаны кодируют в токен-схему. Разные схемы — разные способы обозначить границы.

Схема	Теги	Суть
IO	I, O	самое простое, но две соседние сущности сливаются
BIO (IOB)	B, I, O	B отмечает начало — границы видны; стандарт CoNLL
IOE	I, O, E	то же самое, только отмечается конец, а не начало
BILOU	B, I, L, O, U	добавляются теги конца (L) и одиночного слова (U)
BIES	B, I, E, S, O	максимум классов — сильнее дисбаланс, учить труднее

Общая беда всех токен-схем: вложенные сущности не выразить — «адрес» нельзя разбить на город, улицу и дом одновременно.

РАЗМЕТКА · ВЫБОР

Храним спаны, учим на BIO

ВЫВОД Спаны — универсальный формат хранения и оценки: любой модельный ответ приводится к ним. BIO — формат обучения. Держим оба.

Почему именно BIO

- 1 Классы — полные теги (B-BRAND, I-ATTR и так далее). Буквы B, I, O лишь кодируют границы.
- 2 У 27% запросов сущности длиннее одного токена. Схема IO склеила бы соседние сущности в одну, BIO их разделяет.
- 3 BIOU на коротких запросах даёт больше классов без выигрыша — дисбаланс растёт, качество нет.
- 4 Наши теги: BRAND, MODEL, CATEGORY, ATTR (с приставками B и I) плюс O.

Пример BIO-разметки

ноутбук **B-CATEGORY**

asus **B-BRAND**

zenbook **B-MODEL**

16 **B-ATTR**

гб **I-ATTR**

MVP · АРХИТЕКТУРА

Гибридный каскад: MVP на тот день

КАК УСТРОЕНО К концу дня 03 каскад выглядел так. Каждый слой закрывает слабость предыдущего: правила мгновенные, CRF понимает морфологию, дальше три классификатора — бренд, категория и тип атрибута. На выходе факты за десятки миллисекунд.



ПЕТЛЯ КАЧЕСТВА неуверенные ответы и ошибки уходят в ручную разметку, на ней дообучаем CRF и пополняем словари

ПРОДАКШЕН · ПУТЬ ЗАПРОСА

Путь запроса от строки до выдачи

ЗАЧЕМ Извлечь факты — только середина пути: дальше нормализация, каталог и ранжирование.



Шаг	Пример
Нормализация	«16гб», «16 gb», «16 ГБ» — всё превращается в одно значение: 16 ГБ
Каталог	категория «ноутбук» находит узел «Ноутбуки и планшеты»
Ранжирование	карточки с брендом asus и памятью 16 ГБ идут наверх

MVP · КЛАССИФИКАТОР БРЕНДА

Silver для бренда: 67 классов, не только бренды

КЛЮЧЕВАЯ ИДЕЯ Классификатор — запасной слой, когда словарь не нашёл бренд: на «холодильник» нельзя тыкать в частый клик-бренд. Поэтому в классы добавили NO_BRAND и UNKNOWN.



- 1 Из 63 тысяч агрегированных запросов в silver попали 28 664: у остальных нет надёжного сигнала — их честно выбросили.
- 2 Уверенность метки — из кликов: клик выше в выдаче весит больше, майор-бренд должен набрать не меньше 55% веса.
- 3 NO_BRAND — запрос-категория без бренда. UNKNOWN — бренд есть, но вне топ-65 («стинол», «darina»): модель учится отказываться, а не угадывать похожий.
- 4 Разбиение: 22 846 на обучение, 5 710 на валидацию.

MVP · КЛАССИФИКАТОР БРЕНДА

Четыре модели: что и как обучали

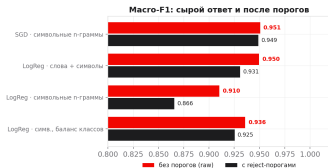
ПОСТАНОВКА Мультикласс запрос → бренд на silver. Все четыре — линейные модели на TF-IDF: отвечают за миллисекунды, критично для SLA. Разница — в признаках и балансировке.

Модель	Признаки	Особенность
LogReg слова+симв.	слова 1-2 + символьные n-граммы 2-4	выбрана в прод: меньше всего ложных брендов
SGD символьная	символьные n-граммы	лучший macro-F1, но чаще врёт на категориях
LogReg символьная	символьные n-граммы	базовый вариант без словных признаков
LogReg с балансом	символьные n-граммы	веса классов — хуже на редких брендах

- 1 Символьные кусочки по 2-4 буквы делают «айфон», «afon» и «iphone» похожими — опечатки не страшны.
- 2 Веса по уверенности клика: сомнительные метки влияют меньше. Все модели сохранены, сравнение — на одной валидации.

MVP · КЛАССИФИКАТОР БРЕНДА

Результаты: 0,95 macro-F1 и честные отказы

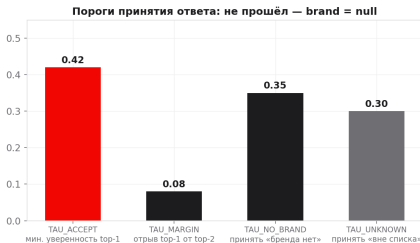


КАК ЧИТАТЬ Верх — macro-F1 до и после порогов отказа. Низ — доля запросов-категорий, где модель ошибочно навязала бренд (главная метрика, цель — ноль).

- 1 По macro-F1 SGD формально впереди (0,951 против 0,950), но разница — третий знак.
- 2 Решает вторая метрика: у LogReg слова+символы ложный бренд на категориях 6,9% против 9,4% у SGD — её и берём в прод.
- 3 F1 спец-классов: NO_BRAND 0,88, UNKNOWN 0,86 — модель научилась говорить «бренда нет» и «не знаю».
- 4 Примеры: «айфон 15 про» даёт Apple, «холодильник» — NO_BRAND.

MVP · КЛАССИФИКАТОР БРЕНДА

Пороги отказа: лучше пусто, чем ложный бренд



Хвост по классам: PlayStation 0,67 (4 примера),

UNKNOWN 0,86, NO_BRAND 0,88 — редкие классы ждут
золотой разметки.

Когда классификатор вообще зовут:

- 1 Словарь или алиас уже нашли бренд — не зовут.
- 2 Запрос — чистая категория — не зовут, бренд пустой.
- 3 Есть намёк на бренд (алиас, похожее на модель слова) — зовут.

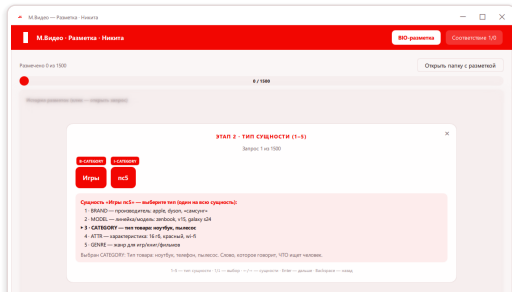
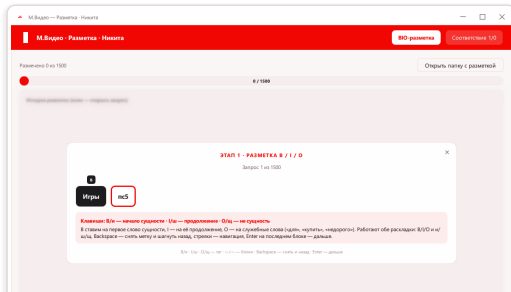
Как принимается ответ:

- 4 Уверенность top-1 не ниже 0,42 и отрыв от top-2 не меньше 0,08 — принимаем бренд.
- 5 Ответ NO_BRAND или UNKNOWN выше своего порога — бренд остаётся пустым, и это правильно.
- 6 Пороги не прошли — отказ, бренд пустой. С порогами точность даже растёт: отказы съедают в основном ошибки.

ПРИЛОЖЕНИЕ · РАЗМЕТКА

Приложение разметки на C++/Qt: мастер из трёх этапов

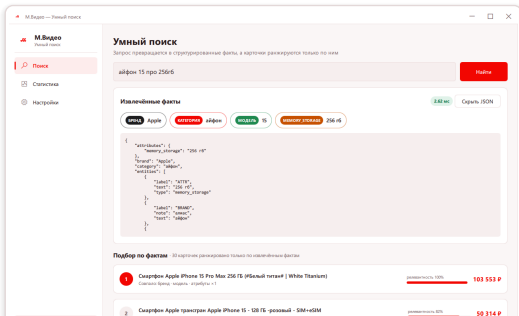
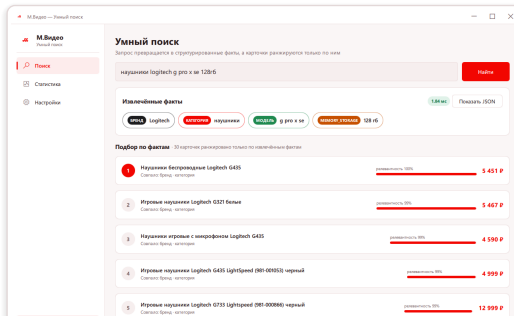
КАК УСТРОЕНО Нативное приложение на C++ и Qt/QML. Запрос разбит на блоки по центру, фон с историей заблюрен. Этап 1: клавиши В/I/O, метка появляется над блоком, Backspace снимает и шагает назад, Enter — подтверждение этапа.



ПРИЛОЖЕНИЕ · УМНЫЙ ПОИСК

MVP умного поиска на C++/Qt

КИЛЛЕР-ФИЧА Ранжирование каталога опирается только на извлечённые факты — бренд, категорию, модель и атрибуты, — а не на сырой текст запроса. Движок на C++ отвечает за 1–2 миллисекунды.



ДЕНЬ 04

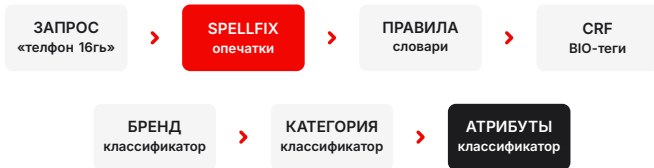
SpellFix, GLiNER и честный каскад

Опечатки до модели, журнал экспериментов, GLiNER zero-shot/FT и joint study на gold/broken.

ПАЙПЛАЙН · ПРОД

Гибридный каскад, который работает сейчас

В ПРОДЕ SpellFix, правила и словари, CRF, затем классификаторы бренда, категории и атрибутов. Тот же каскад в Qt .exe, FastAPI и CLI. GLiNER измерен, в сервис пока не вшит.



- 1 Каждый слой закрывает слабость предыдущего: словарь мгновенный, CRF — морфология, классификаторы — скрытый бренд, категория и тип атрибута.
- 2 Один SpellFix и при сборке silver, и в бою — модель не видит «грязный» текст только в проде.

ДЕНЬ 04 · НОВШЕСТВА

Что изменилось после MVP

КОРОТКО Закрыли шум на входе, завели человекочитаемый журнал экспериментов, прогнали GLiNER как хвост и впервые измерили весь каскад на трёх типах данных.

SpellFix

гомоглифы, fuzzy,
алиасы транслита

CRF rebuild

пересборка silver
и рост gold microF1

Журнал

снимки метрик
и честные кейсы

GLiNER + joint

один silver,
метрика пайплайна

SPELLFIX

Что сделали в SpellFix

Три механики

Механика	Пример
Fuzzy + клавиатура	«16гь» становится «16 гб», «телфон» — «телефон»
Гомоглифы	кириллица и латиница-двойники: с/с, а/а, е/е
Алиасы	«сони» становится «sony», «ксяоми» — «xiaomi»

Эффект на CRF

SpellFix тронул **767 / 5000** silver-запросов до обучения.

Gold micro-F1: **0,582** до правки и **0,594** после.

Сильнее всего вырос ATTR (+0,07) — единицы с опечаткой больше не ломают учителя.

ВЫВОД Одна функция чинит текст и при сборке обучающих данных, и в бою — без отдельного «продакшен-режима».

ДЕНЬ 04 · ЖУРНАЛ

Честные кейсы и журнал экспериментов

SpellFix справляется

телефон 16 гь > телефон 16 гб

планше txiaomi > планшет xiaomi

ноутбук asus 16гь > ноутбук asus 16 гб

Пока нет

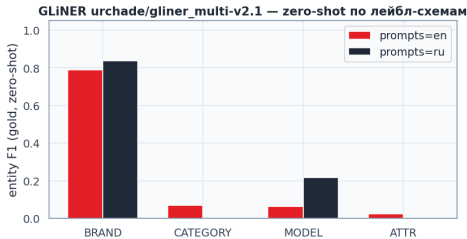
моильник 16 гб — слишком далёкая порча (3 буквы),
вне fuzzy

ЖУРНАЛ Папка снимков метрик CRF, brand и ATTR плюс `broken_queries.md`. Git хранит код; сюда пишем, что поменяли и что получили.

НЕ СПРЯТАЛИ На «sony ps5» NER ок, но классификатор категории иногда путает PlayStation с феном или смартфоном — отдельный баг, не SpellFix.

GLINER · ZERO-SHOT

Что даёт модель без дообучения

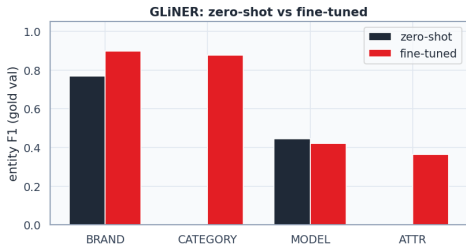


МОДЕЛЬ urchade/gliner_multi-v2.1 — мультязычный чекпоинт. Роль — гибкий хвост, не замена CRF.

- 1 Лучшая схема промпта — английские лейблы: micro-F1 **0,35** на gold.
- 2 BRAND находится и без обучения ($F1 \approx 0,79$).
- 3 CATEGORY / ATTR почти пустые — домену нужны примеры.

GLINER · FINE-TUNE

После обучения на gold + silver



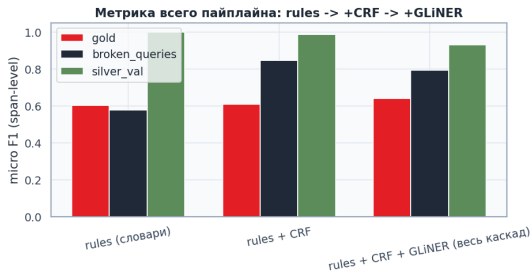
ВЫВОД micro-F1 **0,39** → **0,74**: CATEGORY подтянулась с 0 до 0,88 — модель увидела домен.

- 1 Порог откалиброван отдельно ($\approx 0,35$): default 0,5 резал верные, но неуверенные спаны.
- 2 ATTR остаётся самым слабым ($F1 \approx 0,36$) — мало и шумно в gold.
- 3 В прод не подключён: сначала гейт и latency-бюджет.

JOINT STUDY

Один silver — честное сравнение каскада

ИДЕЯ CRF и GLiNER на **одном** silver; gold — held-out. Стадии: rules, затем +CRF, затем +GLiNER.



- 1 На **broken** главный прирост — CRF: **0,58** → **0,85**.
- 2 На **gold** GLiNER сверху: **0,61** → **0,64**.
- 3 На шуме «слепой» хвост **портит** precision (0,85→0,80).
- 4 Silver-val $\approx 1,0$ — самосогласие teacher, не истина.

ВЫВОД GLiNER звать только когда rules+CRF ничего не нашли — не для безусловного дозаполнения.

МЕТРИКИ · СВОДКА

Честные ориентиры на одной странице

Что	было	стало	где смотреть
CRF gold micro-F1	0,582	0,543	n=823, честный eval
Brand clf macro-F1	—	0,94	silver-val
Каскад gold (rules+CRF+GL)	0,604	0,640	joint study
Каскад broken (rules+CRF)	0,578	0,846	устойчивость к шуму
GLiNER FT gold	0,39	0,74	изолированный трек

Главное правило: *silver-val* красивый, но завышен. Решения принимаем по *gold* (сейчас 823 запроса) и по *broken-кейсам*.

ДЕНЬ 05

Сервис, золотая разметка и скорость

Подключили FastAPI и CLI, дообучили модели на ручной разметке (больше 500 товаров) и замерыли, сколько миллисекунд занимает каждый слой каскада.

ДЕНЬ 05 · FASTAPI

HTTP API и CLI для того же каскада

ОДИН EXTRACTOR Qt .exe, FastAPI и CLI ходят в одну логику: SpellFix → правила → CRF → brand/ATTR. Меняем слой один раз — проверяем тремя способами.

FastAPI (src/service)

- 1 POST /extract — JSON фактов для интеграции.
- 2 POST /extract/debug — BIO-пайплайны для лаборатории.
- 3 GET /health — готовность моделей и словарей.
- 4 Веб-UI из web/ на том же порту.

CLI

- 1 `python -m src.cli.cli " " — smoke-тест каскада.`
- 2 Режим `-i / --demo` для демо без UI.
- 3 Тот же `QueryEntityExtractor.from_artifacts.`

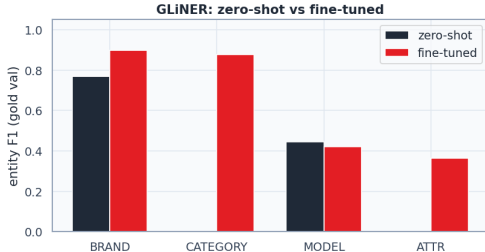
ДЕНЬ 05 · GOLD

Дообучение на золотой разметке

ВЫВОД Уже размечено **более 500 товаров** под gold. GLiNER fine-tune на gold+silver: micro-F1 **0,39** → **0,74** на gold-val.

Что сделали

- 1 Gold держим как held-out: не мешаем с silver-учителем.
- 2 Fine-tune GLiNER (urchade/gliner_multi-v2.1) на 154 gold + silver.
- 3 Порог откалиброван ($\approx 0,35$) — default 0,5 резал верные спаны.
- 4 Разметка — в MvLabel (три аккаунта команды).



ДЕНЬ 05 · GOLD EVAL

Честный инференс на gold: 823 запроса

ЧТО СДЕЛАЛИ Прогнали текущий каскад на расширенном gold (bio_liza.jsonl, n=823) без дообучения. Это не silver-val: здесь видно реальный потолок.

Слой	micro-F1	BRAND	CATEGORY	ATTR
rules (словари)	0,533	0,81	0,58	0,28
CRF (прод)	0,543	0,79	0,60	0,29

- 1 На полном gold CRF чуть выше правил (0,533 → 0,543): бренд держится, MODEL и ATTR всё ещё слабые.
- 2 Согласие типов атрибутов с gold: teacher 0,43, классификатор 0,54 — запасной слой ATTR полезен, но gold ещё шумный.
- 3 Раньше на срезе ~200 запросов gold micro-F1 выглядел ~0,59; на 823 честнее и жёстче — решения принимаем по этой цифре.

ДЕНЬ 05 · LATENCY

Скорость каскада: где укладываемся в SLA

ОРИЕНТИР Прод-Qt / rules+CRF — десятки мс. GLiNER как хвост — ~250 мс/запрос: поэтому в прод только с гейтом, не на каждый запрос.

Стадия (gold, n=181)	F1	latency	где
rules (словари)	0,604	21 мс	голова
rules + CRF	0,609	15 мс	прод-каскад
+ GLiNER (хвост)	0,640	249 мс	эксперимент
C++ MvSearch .exe	—	1–2 мс	десктоп

1 На **broken_queries** CRF даёт главный прирост (0,58→0,85) при ~23 мс — устойчивость к шуму без трансформера.

2 GLiNER zero-shot / FT: ~190–210 мс на запрос — ок для офлайн-хвоста, не для SLA <100 мс на всём трафике.

ДЕНЬ 05 · РЕПОЗИТОРИЙ

Код, релизы и презентации

GITHUB Весь пайплайн в одном репозитории:

[github.com/xWooshieL/
mvideo-ner-search](https://github.com/xWooshieL/mvideo-ner-search)

Releases: v0.4.0-final

SmartSearch Setup 0.4.0 + Labeling 0.1.4

+ итоговая презентация

Что внутри

- 1 C++/Qt приложения + Inno Setup.
- 2 Python: CRF, brand_clf, SpellFix, FastAPI, CLI.
- 3 Ноутбуки: EDA, GLiNER, general_study.
- 4 Журнал экспериментов со снимками метрик и broken-кейсами.

Спасибо!

Вопросы по пайплайну, SpellFix, GLiNER и метрикам — welcome.

Команда:

Новицкий Никита · @n.novitskiy

Борисов Никита · @n.borisov

Засухина Елизавета · @e.zasukhina

Олейников Александр · @a.oleynikov